

V1

REAL PROBLEM

2 - MVCC, VACUUM, BLOAT AND TRANSACTION ID WRAPAROUND

the single most important operational topic - [MID](#) why bloat happens - [STAFF](#) the failure that stops the database

- **HOT updates** avoid touching indexes when no indexed column changes and the new version fits on the same page — a large win, and one reason to drop unused indexes
- **Lower fill factor** on heavily updated tables leaves room on each page for HOT updates
- **Batch large deletes**, or use partitioning so removal is dropping a partition rather than deleting rows

DEPLOYMENT TOPOLOGIES – what each one gives you, and what

[illegible]

up restored, and what is the largest table's unfrozen age?

- Queries are compressed and, if table is transposed
- Writes, unions and digests
- Use `SELECT * FROM table` with selecting the columns you need
- Tables why a table with a rowid column
- It is usually the wrong call

JUNIOR B-tree covers most cases · **MID** the specialised types · **STAFF** what indexes cost

Indexes are not free, and the cost tends to be on you. Every index must be updated on insert and delete, and on any updates that change the columns – or that cannot be performed as a HOT update. Each one also consumes cache and needs vacuuming. **An unused index is a permanent tax on every write** with no compensating benefits.

Find unused indexes with `pg_stat_user_indexes`. Scan counts near zero after a full business cycle means the index is not earning its cost. Drop them – but check replicas too, since a reporting metric may be the only user.

Index-only scans

The fastest read possible: the query is answered from the index without touching the heap. It requires **every requested column to be in the index** and the page to be marked all-visible in the visibility map – which is maintained by vacuum. **A poorly vacuumed table silently loses index-only scans**, and the query gets slower with no change to the code.

Building and maintaining these indexes is more work:

- CREATE INDEX CONCURRENTLY** to production. A plain `CREATE INDEX` takes a lock that blocks writes for the whole build
- It is slower and can fail**, leaving an invalid index behind that must be dropped and retried – check these afterwards
- REINDEX CONCURRENTLY** requires a blocked index without the long lock
- Indexes show too**, particularly under heavy updates – monitor size against table size

Before adding an index, check whether **one already serves** the query. A multicolumn index on (a, b) already handles queries on a, so adding a separate index on a adds write cost for nothing. Duplicated and overlapping indexes add needless query overhead.

MID physical versus logical - **STAFF** what core does not provide, and the classic outage

The classic PostgreSQL, *outageless*, or *inactive* replication slot. A slot guarantees the primary retains WAL, until the consumer has it — which is exactly what you want while the replica exists. But if the replica is decommissioned, or at logical storage steps consuming, the slot remains and WAL accumulates without finish until the disk fills and the database stops. It is *indefinitely* alive until it is critical.

The prevention is three times of monitoring. Alert on any slot where active is low, on the related WAL size per slot, and on the WAL replication as a proportion of the volume. Set `max_slot_wal_keep_size` to a stock slot is invalidated rather than allowed to take the primary down — losing a replica is much better than losing the primary.

High availability needs tuning

Core PostgreSQL will not promote a replica for you. Streaming replication gives a standby; something else must decide the primary is gone and act

- Patrol with `pg_ctl` or `Corral` is the common choice — *quorum-based* leader election with standby to prevent two primaries
- reperage** and **autoheal** are alternatives with less coordination machinery
- Client must find the new primary** via a virtual IP, a proxy such as HAProxy or PgBouncer, or multi-host connection strings with target session attributes

Test failover deliberately, including that the application recognizes and the old primary cannot back as a second writer

Replicas are not a backup. A mistaken DELETE or a bad migration replicates in milliseconds. Only backups with point-in-time recovery let you go back to just before the mistake — see panel 13.

10 • PARTITIONING what it does and does not fix

Partitioning partitions splits one logical table into physical partitions by a key. The application queries the parent table and PostgreSQL routes reads and writes to the relevant partitions.

Strategy	Use for
Range	The common case — by date, for time-series data with retention
List	Discrete values — region, tenant, category
HASH	Even distribution when no natural key exists

What it genuinely buys

- Retention becomes DETACH or DROP of a partition — near-instant, and no block. Compare with deleting millions of rows, which creates dead tuples and demands vacuum
- Partition pruning — sparse filtering on the key skips irrelevant partitions entirely, at plan time or execution time
- Maintenance operations partition — vacuum, analyze and index builds work on manageable pieces
- Block loading into a fresh partition, instead forwards and then attached

What it does not buy: It is **not a general performance fix**. A query that does not filter on the partition key must touch **every** partition, and is usually slower than it would have been against one table. Partitioning solves manageability and retention first, and query performance only for key-aligned access.

The constraints to plan around

- Unique constraints must include the **partition key** — you cannot enforce global uniqueness on another column
- Foreign keys referencing a **partitioned table** have historically been restricted; control behavior for your version
- No **many partitions across planning time** — thousands is a real number. Match granularity to retention
- Partitions must be created ahead of time** — `pg_dump` train or a scheduled job. A missing partition means failing inserts, usually at midnight

STAFF what each one does, and the one that is per-operation rather than per-server

yet, it's the setting most often misunderstood, and it's worth being precise about. It is the hint for each individual task, hash or timer operation. A single query with several ports and a hash join can allocate it several times over, and every connection needs the same independently. Total consumption is roughly `num_tasks * operations * connections` – which is how a value that looked reasonable turns a machine out of memory under load.

The safe approach: keep the global value conservative and raise it for the specific session, role or query that genuinely needs it. A reporting job can set it high for itself without putting the whole server at risk.

Setting work on setting on day one

```
statement_timeout = 0; -- no query should run forever. Set it per role, higher for reporting than for the application
* idle_in_transaction_session_timeout = 0; prevents the most common cause of unbounded block
log_timeout = 0; set before DDL so a schema change cannot go the world behind it
log_min_duration_statement = 0; log slow queries. The cheapest observability available
log_checkpoints, log_lock_waits, log_autovacuum_min_duration = 0; -- turns a silent problem into a log line
shared_preload_libraries = 'pg_stat_statements'; requires a restart, so do before you need it
```

View	Answers
pg_stat_statements	Which queries consume the most total time – usually more useful than the single slowest
pg_stat_activity	Which is running right now, what is waiting, and how long the oldest transaction has been open
pg_locks	What is holding what and who is blocked. Join to activity to find the head of a lock queue
pg_stat_user_tables	Dead tuple counts, and when each table was last vacuumed or analyzed
pg_stat_user_indexes	Index scan counts – how you find indexes nobody uses
pg_stat_replication	Replica state and lag, in bytes and in time
pg_replication_slots	Inactive slots and retained WAL . Check this before it fills a disk
pg_stat_database	Commits, rollbacks, deadlocks, temporary file usage and cache hit ratio

Temporary file usage is an underused signal. It means operations spilled to disk because work_mem was insufficient – a precise, actionable pointer to which queries need reordering, rather than a general sense that things are slow.

	Symptom	Cause	Fix
Application slow, database looks idle	Connection pool exhausted; requests queuing before they reach the database		Monitor pool, add a pool, or increase holding count
Queries starting to fail	Out of memory under load	work_mem multiplied by operations and connections	Lower the value or plan more carefully
Queries timing out	Regular write latency spikes	Checkpoints too frequent, or not spread across the interval	Raise max_wal_size, check the "checkpoint_timeout" parameter, or increase the interval
Replica falls behind and cannot catch up	Replica is single-threaded and the primary's write rate exceeds it		Reduce write rate, faster replication, or accept that this is a limitation
Queries on the replica cancelled	Replica conflicts with a long-running read		hot_standby = on known; back up on primary

- Unused indexes identified and dropped
 - Indexes built with **`CONCURRENTLY`**: invalid ones cleared up
 - `random_page_cost` lowered for **`SSD`**
 - shared buffers and effective cache size sized
 - work_mem conservative globally: raised per session where needed
 - Temporary file usage monitored as a work_mem signal
 - Checkpoint frequency tuned: log checked for frequent checkpoints
 - Connection pooler in place: txn_connections_max modest
 - Pool mode understood — what transaction mode breaks
 - Pool wait time monitored on the application side
 - Replication log monitored in bytes and seconds
 - Replication slots checked for active ones: txn_slot_val_keep_size set
 - WAL directory disk usage alerted on separately
 - Failing toolbar dropped — core has none

backup was last restored and tested. Those four answers surface the failures that actually take PostgreSQL down — none of which are about query throughput, and all of which are visible weeks later.

start here — MVCC and the process-per-connection model explain almost everything else on this sheet

The pragmatic middle ground is Read Committed with explicit `SELECT ... FOR UPDATE` on the rows an invariant depends on. That takes real row locks and is often simpler to reason about than raising the isolation level globally.

3. LOGICAL REPLICATION Publish and describe at table granularity, decided from the WMS, into the changes. Solve: selecting dimensional, cross-system or logical replication. It is one-dimensional. Dimensional source create conflicts must be resolved prior – this is not material master, and should not be sold as it is.	3. MANAGED HIGH PRIORITY The same replication, just Palmer, memory or data. Release to shared table and promote. Solve: distributed table with a real cursor. Save a distributed table – direct or Cursor – and a table or array to table for table for table. This is the production answer. Streamline replication store is not high available.	4. LOGICAL REPLICATION Publish and describe at table granularity, decided from the WMS, into the changes. Solve: selecting dimensional, cross-system or logical replication. It is one-dimensional. Dimensional source create conflicts must be resolved prior – this is not material master, and should not be sold as it is.
4. LOGICAL REPLICATION Publish and describe at table granularity, decided from the WMS, into the changes. Solve: selecting dimensional, cross-system or logical replication. It is one-dimensional. Dimensional source create conflicts must be resolved prior – this is not material master, and should not be sold as it is.	4. LOGICAL REPLICATION Publish and describe at table granularity, decided from the WMS, into the changes. Solve: selecting dimensional, cross-system or logical replication. It is one-dimensional. Dimensional source create conflicts must be resolved prior – this is not material master, and should not be sold as it is.	4. LOGICAL REPLICATION Publish and describe at table granularity, decided from the WMS, into the changes. Solve: selecting dimensional, cross-system or logical replication. It is one-dimensional. Dimensional source create conflicts must be resolved prior – this is not material master, and should not be sold as it is.

The planner is cost-based and driven by query cost. It estimates rows at each step and picks the cheapest plan. It has **no memory of previous executions** — so a plan changes when the statistics change, which is why a query can turn slow overnight with no deployment.

```
EXPLAIN (ANALYZE, BUFFERS) SELECT --
-- ANALYZE actually runs it, giving row linkings
-- BUFFERS shows pages hit vs read from disk
-- The key signal:
Seq Scan on orders
(costs=rows|IO) (actual = rows|IO|IO)
estimated |
actual |
-- a large one means the statistics are wrong,
-- and every choice above this node is suspect
```

Reading the plan nodes

Seq Scan	Reads the whole table. Correct for small tables or when returning most rows — not automatically a fault
Index Scan	Walks the index, then fetches rows from the heap
Index Only Scan	Answered entirely from the index. The fastest case
Bitmap Heap Scan	Closest matches from the index, then reads the heap in physical order. Efficient for medium-sized result sets
Nested Loop	When one side is small, disastrous when the row estimate was wrong and it is actually large
Hash Join	Builds a hash of one side. Needs work, not a split
Merge Join	Both inputs sorted — good for large, already-ordered sets

The first thing to look for is not the slowest node — it is the largest gap between estimated and actual rows. The planner made every subsequent choice on that estimate. Fix the estimate, with ANALYZE, a higher statistics target, or extended statistics for correlated columns, and the plan often corrects itself.

work is as sort or hash operation, not per query or per connection. A query with several sorts can sail several times faster, and many concurrent connections multiply that again. Getting it slightly higher is a well-known route to turning the machine out of memory — as it is for the specific session and query that needs it.

not optional at scale

Each connection is a separate operating-system process with its own memory and scheduling cost. Idle connections still consume resources. This is why `max_connections` set to a high number is a symptom, not a solution – a database configured for a thousand direct connections will spend a large share of its time on just connecting and/or context switching rather than on queries.

The shape of the right answer: keep `max_connections` low – commonly in the low hundreds – and put a pooler in front to multiply many application connections onto few database ones. Applications open connections to the pooler, which is cheap; the database sees a small, steady number.

Bouncer pool modes	
Mode	Behaviour
Session	A server connection is held for the client's whole session. Stafest, least pooling benefit
Transaction	Returned to the pool after each transaction. The usual production choice – very high multiplexing
Statement	Returned after every statement. Multi-statement transactions are impossible

Transaction mode breaks anything that assumes a persistent session. Because a different backend may serve your next transaction: **session-level SET, advisory locks, LISTEN/NOTIFY, SETS, HOLD cursors and temporary tables** all misbehave. Server-side prepared statements need matching support on both sides. These features are intermittent and confusing precisely because they depend on which backend they land on.

Getting the sizing right

The database pool should be small – a common starting point relates it to CPU cores rather than to application instances

Application pools must sum to less than the pooler's limit, or connections queue in the wrong place

`app_in_transaction_session_timeout` so a leased transaction cannot hold a server connection and block vacuum

Reuse expensive connections so the administrator can get them in when the pool is exhausted

Connection exhaustion usually presents as a slow application, not a database error. Requests queue waiting for a connection while the database itself looks idle. Monitor pool wait time on the application side – the database will not tell you.

how a schema change causes an outage

The mechanism that surprises people: locks *queue*, and a waiting lock blocks *everything* behind it. An ALTER TABLE needing an ACCESS EXCLUSIVE lock waits for a long-running query to finish – and every subsequent query that table queues behind the waiting ALTER, including simple reads that would otherwise have been fine. A one-second schema change becomes a total outage on that table.

The *defence* is one line: Set LOCK_TIMEOUT before any DDL, so the statement gives up quickly rather than accumulating a queue behind it. Retry in a loop if needed – failing fast repeatedly is safer than waiting once.

– the safe pattern for any production DB:
SET LOCK_TIMEOUT = '30s';
ALTER TABLE orders ADD COLUMN new_text;
– fails fast (instead of queuing the whole world it)

Operations and what they take

	Fast – metadata only since PostgreSQL 11. It does not rewrite the table
ADD COLUMN, nullable or with a constant default	
ADD COLUMN with a volatile default	Rewrites the whole table under an exclusive lock
Changing a column type	Usually a full rewrite. Prefer a new column, backfill, then swap
SET NOT NULL	Scans the table. Add a validated CHECK constraint first to make it cheap
ADD INDEX (VOLTD, then VALIDATE separately)	– the validation takes a much weaker lock Blocks readers. Always use CONCURRENTLY on production
CREATE INDEX	Blocks everything. Use <code>idx_repack_index()</code> instead
Drop	

Drop *is not* distributed and needs a coordinated shutdown. PostgreSQL doesn't even begin a drop operation with a table open. They use some clever cleverness to have made people follow the correct behaviour.

and the difference between them

Access control

- pg_hba.conf is the first gate** – it decides who may connect, from where, and how. It is evaluated top to bottom, first match wins, and it is where accidental trust entries hide
- Use `scram-sha256`** – not the oldest md5 method
- Roles, not users** – group to group roles and make people and services members
- Never let the application connect as a *superuser* or as the *table owner*** – the owner can drop the table; the application only needs to read and write them
- Row-level security** enforces per-row visibility in the database – the mechanism for multi-tenancy
- TLS for all connections**, and verify the certificate on the client rather than merely enabling encryption

Revoke the default public schema rights on new databases

Backups – three different things

- pg_dump** Logical. Portable across versions and architectures, restores selected objects, and is **slow** to restore at large sizes
- pg_basebackup** Physical. A byte-wise copy of the cluster – fast to restore, but can't drop the table
- WAL** A physical base backup **plus continuously archived WAL**, allowing recovery to any moment. **The only thing that undergoes a big migration at 16:32**

Replication is not backup, it faithfully copies your mistakes. Point-in-time recovery is what lets you go back to just before one – and it's the difference between an incident and a catastrophe.

Use `pgwal-e` or `harnan` rather than hand-rolled scripts. They handle parallelism, compression, retention, and – importantly – **verification**. Then restore from them on a new machine and time: it's an untested backup is a hypothesis, and restore duration is the number your objective actually depends on.